

Laboratoria: piątek 13:30
Grupa L12
Informatyka

Algorytmy i Struktury Danych
Prowadzący:

Dominik Witczak

Sprawozdanie do

Ćwiczenia 1 – Algorytmy sortowania (29.03.2024)

Autorzy:

Mateusz Kaliński
160354

Mateusz Kaźmierczak
160162

1 Wprowadzenie

1.1 Insertion Sort

Insertion Sort (sortowanie przez wstawianie) jest prostym algorytmem, który działa w sposób iteracyjny poprzez przechodzenie przez tablicę danych i porządkowanie elementów.

Początkowo ciąg dzielony jest na część **posortowaną** i część **nieposortowaną**. Elementy są przenoszone wewnątrz tablicy, aż do znalezienia swojego właściwego miejsca.

1.2 Selection Sort

Selection Sort (sortowanie przez wybieranie) to również prosty algorytm, który działa przez iteracyjne wyszukiwanie najmniejszego (lub największego) elementu z nieposortowanej części tablicy i zamianę go z elementem na początku nieposortowanej części.

1.3 Heap Sort

Heap Sort (sortowanie przez kopcowanie) to efektywny algorytm, który wykorzystuje strukturę danych kopca do sortowania elementów. Działanie tego algorytmu polega na tworzeniu kopca z danych wejściowych, a następnie usuwaniu elementów z kopca w odpowiedniej kolejności, co prowadzi do posortowanej tablicy.

1.4 Shell Sort

Shell Sort (z przyrostem Sedgewicka) to algorytm, który jest ulepszoną wersją sortowania przez wstawianie.

Algorytm dzieli tablicę na mniejsze podtablice, a następnie sortuje te podtablice za pomocą sortowania przez wstawianie. W miarę postępu algorytmu, odległości między elementami, które są porównywane i zamieniane, są zmniejszane, co pomaga w przesuwaniu mniejszych elementów na początek tablicy szybciej.

Przy wykonywaniu algorytmu wykorzystaliśmy przyrost Sedgewicka, który zaimplementowaliśmy następująco:

```
while gap < len(arr) // 3:
    index += 1
    gap = (4**index + 3 * 2 ** (index-1) + 1)
    gaps.insert(0,gap)
```

1.5 Quick Sort

Quick Sort to efektywny algorytm, który używa strategii "dziel i zwyciężaj" do sortowania elementów w tablicy.

Quick Sort wybiera element referencyjny z tablicy (nazywany pivotem) i dzieli tablicę na dwie podtablice: jedną zawierającą elementy mniejsze niż pivot i drugą zawierającą elementy większe niż pivot. Następnie rekurencyjnie sortuje obie podtablice.

Przy pisaniu algorytmów korzystaliśmy z lewego pivota oraz z pivota losowego. Losowy pivot pomaga zniwelować potencjalne niekorzystne warunki, które mogą wystąpić przy stałym wyborze pivota, dlatego jest on częściej rekomendowany.

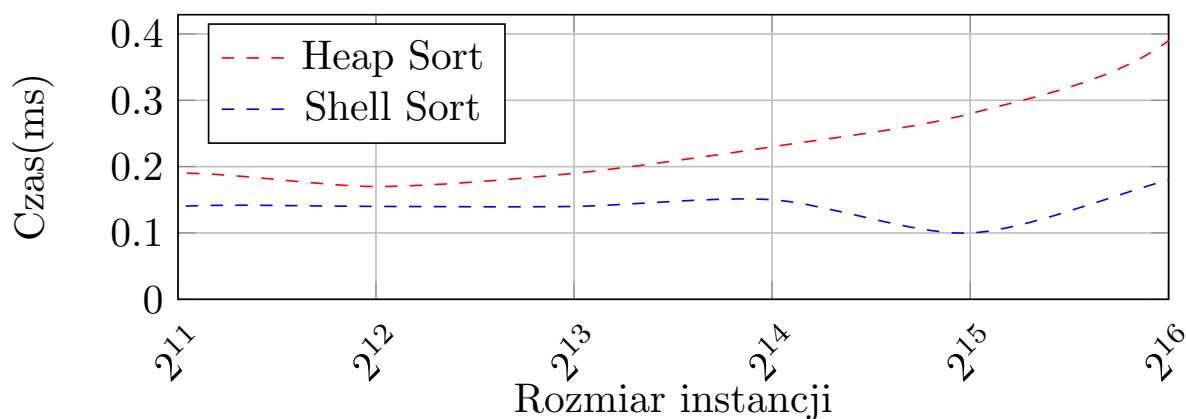
2 Złożoność czasowa - porównanie między algorytmami

Il.liczb	HeapSort	InsertionSort	QuickLeft	QuickRandom	SelectionSort	ShellSort
4	0.01	0.00	0.00	0.00	0.01	0.01
8	0.00	0.01	0.00	0.01	0.01	0.01
16	0.01	0.01	0.00	0.01	0.01	0.01
32	0.00	0.01	0.00	0.01	0.01	0.01
64	0.00	0.01	0.00	0.01	0.01	0.01
128	0.01	0.01	0.01	0.01	0.01	0.01
256	0.01	0.01	0.00	0.01	0.01	0.01
512	0.01	0.01	0.01	0.01	0.01	0.01
1024	0.01	0.03	0.01	0.01	0.03	0.01
2048	0.01	0.10	0.01	0.01	0.09	0.01
4096	0.02	0.39	0.01	0.01	0.35	0.02
8192	0.04	1.52	0.02	0.02	1.40	0.02
16384	0.07	6.16	0.03	0.04	5.57	0.05
32768	0.16	25.34	0.06	0.07	22.78	0.10
65536	0.35	109.09	0.14	0.12	94.33	0.22
131072	0.76	510.55	0.30	0.23	404.00	0.50

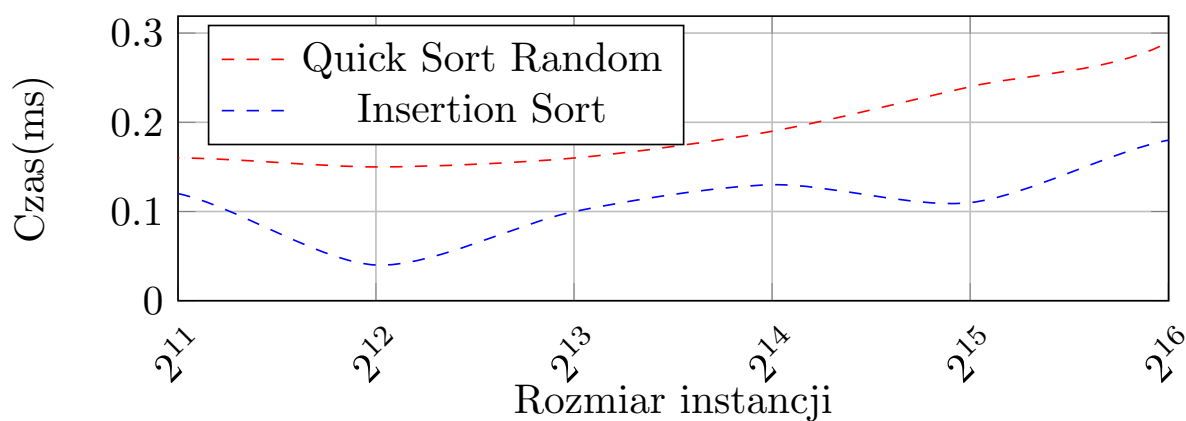
3 Złożoność czasowa - optymistyczna i pesymistyczna

3.1 Dane rosnące

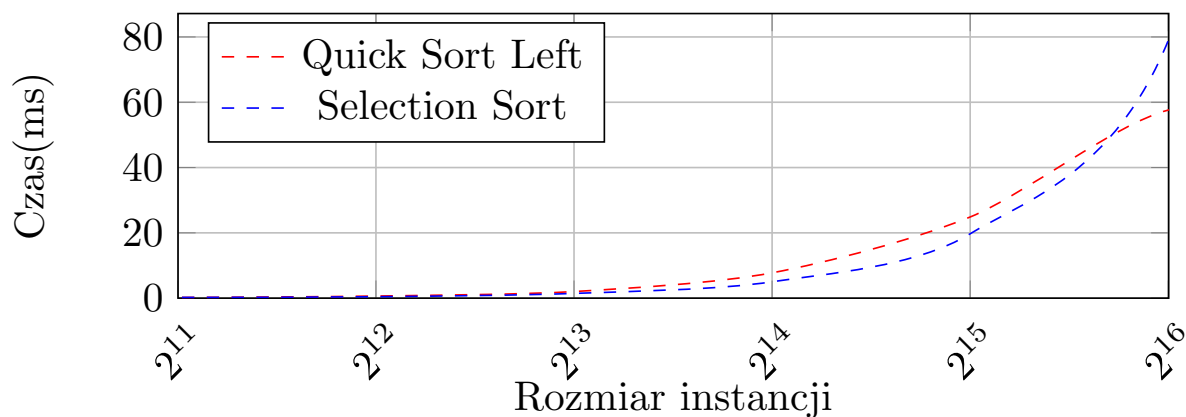
Złożoność Obliczeniowa Algorytmy Szybkie



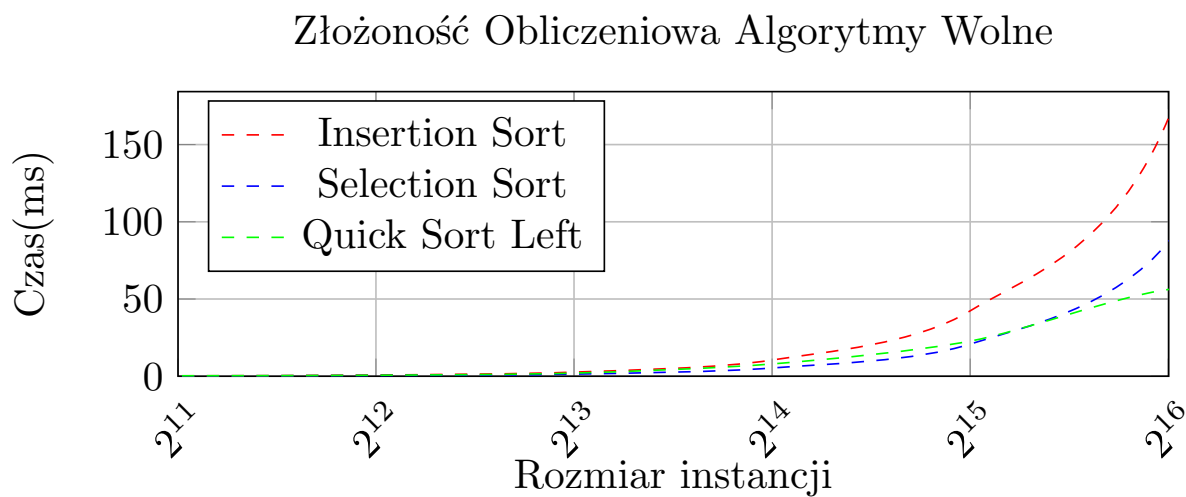
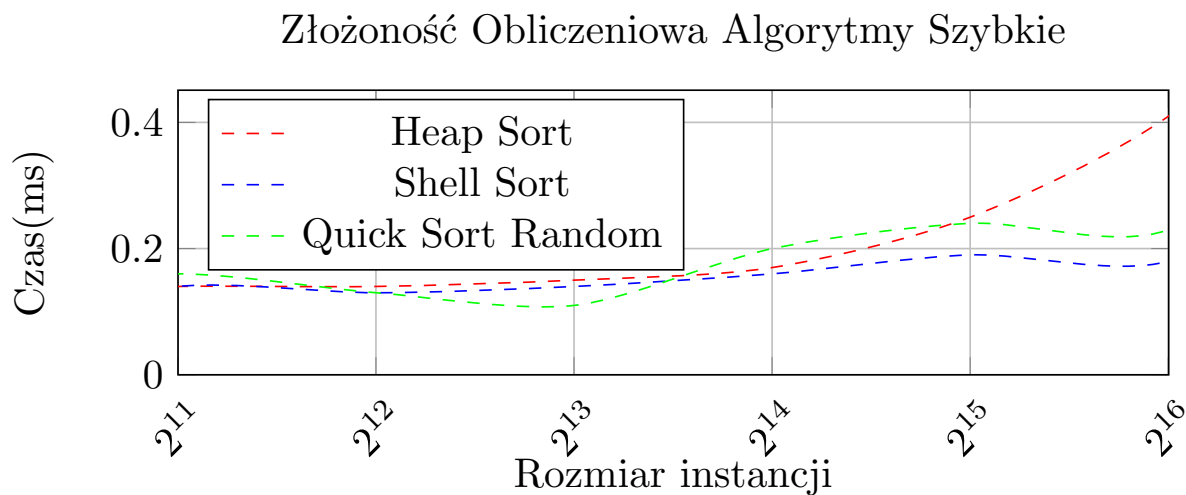
Złożoność Obliczeniowa Algorytmy Szybkie



Złożoność Obliczeniowa Algorytmy Wolne

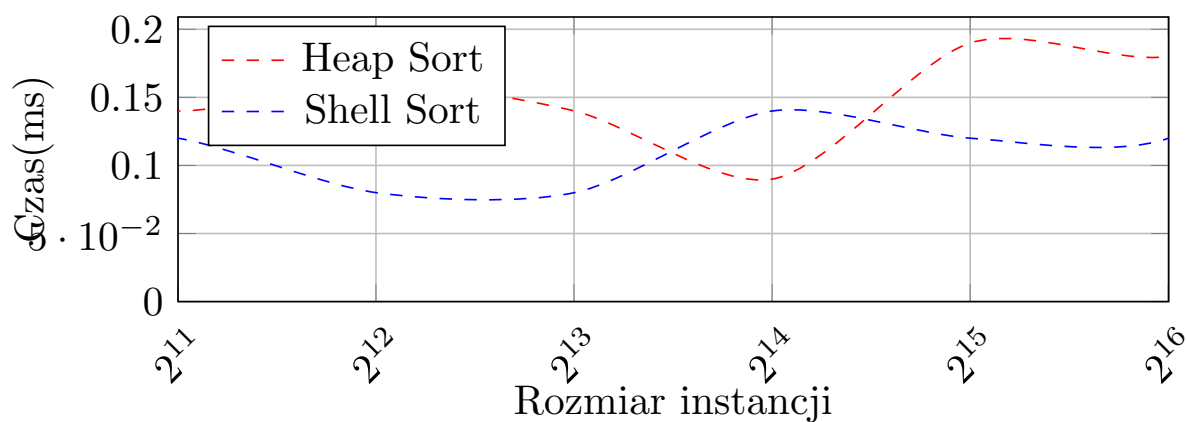


3.2 Dane malejące

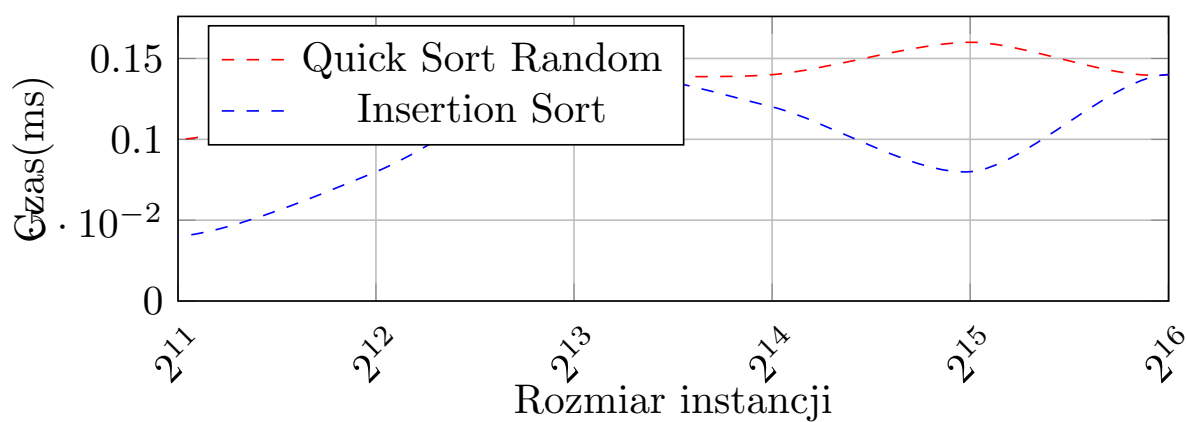


3.3 Dane stałe

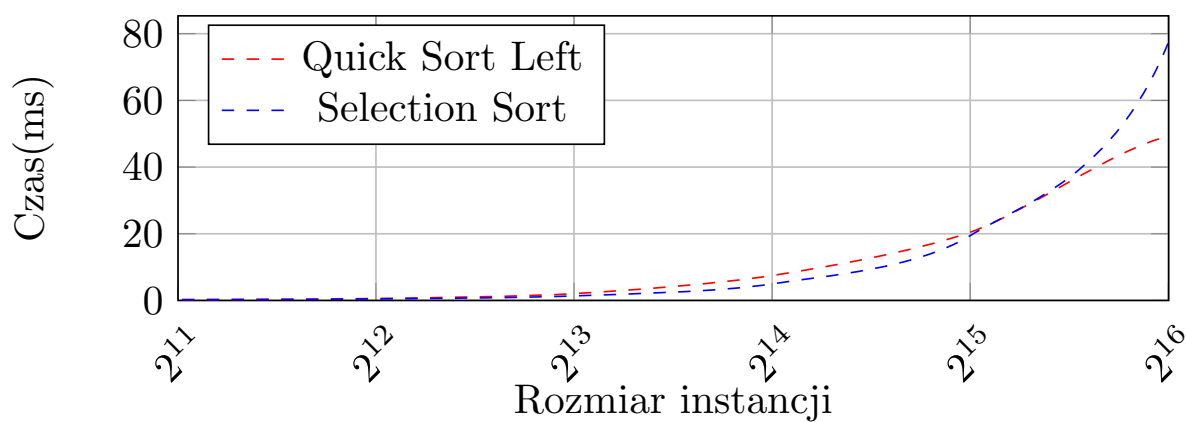
Złożoność Obliczeniowa Algorytmy Szybkie



Złożoność Obliczeniowa Algorytmy Szybkie

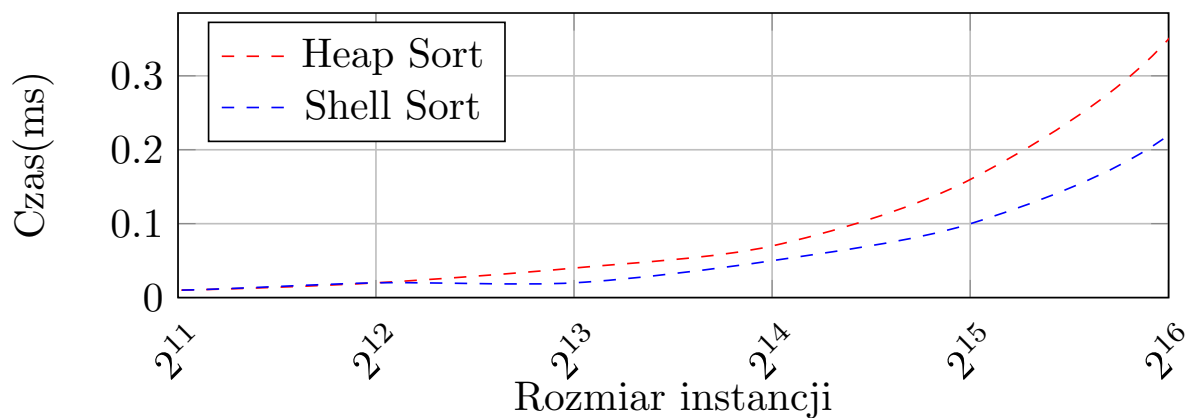


Złożoność Obliczeniowa Algorytmy Wolne

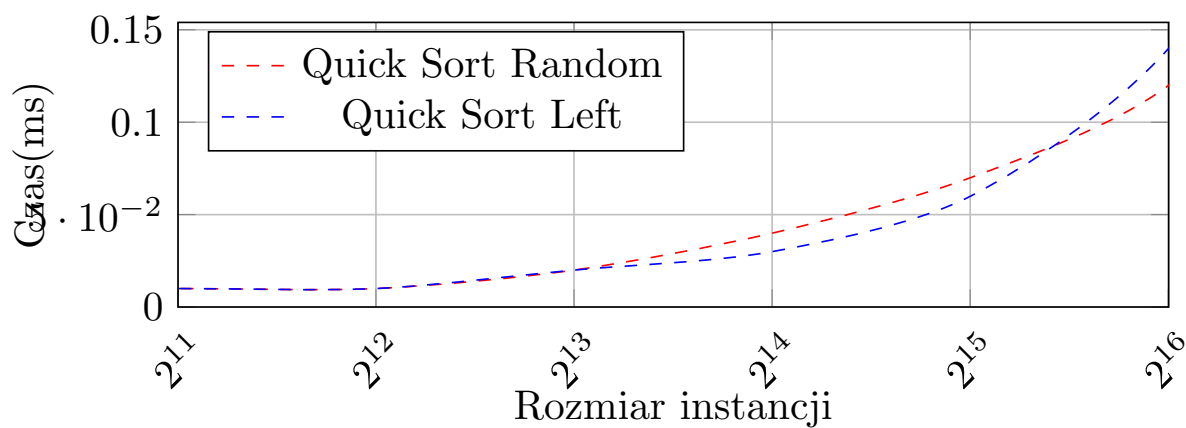


3.4 Dane losowe

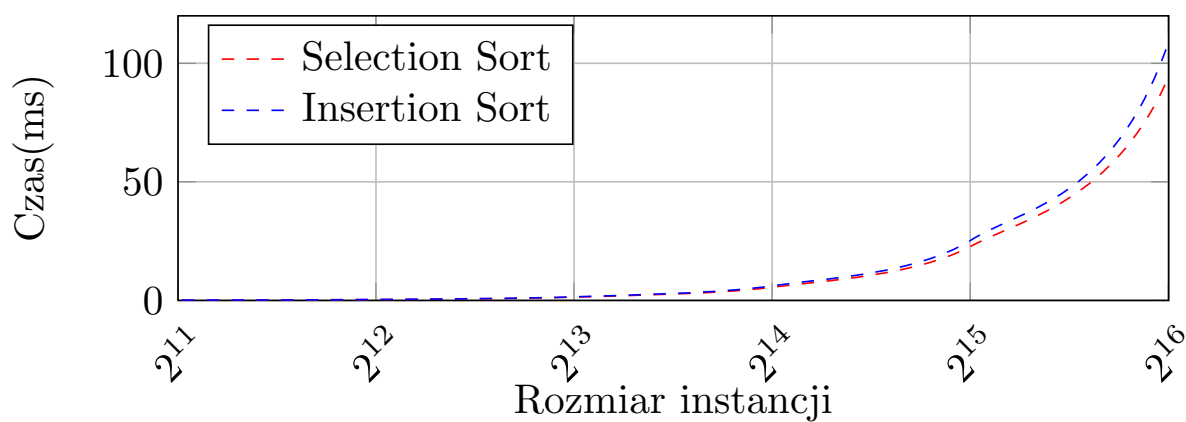
Złożoność Obliczeniowa Algorytmy Szybkie



Złożoność Obliczeniowa Algorytmy Szybkie

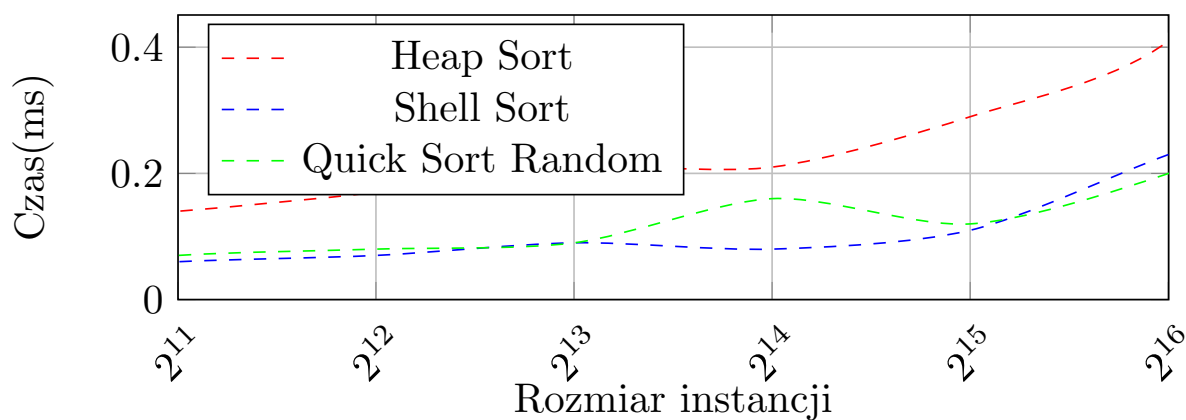


Złożoność Obliczeniowa Algorytmy Wolne

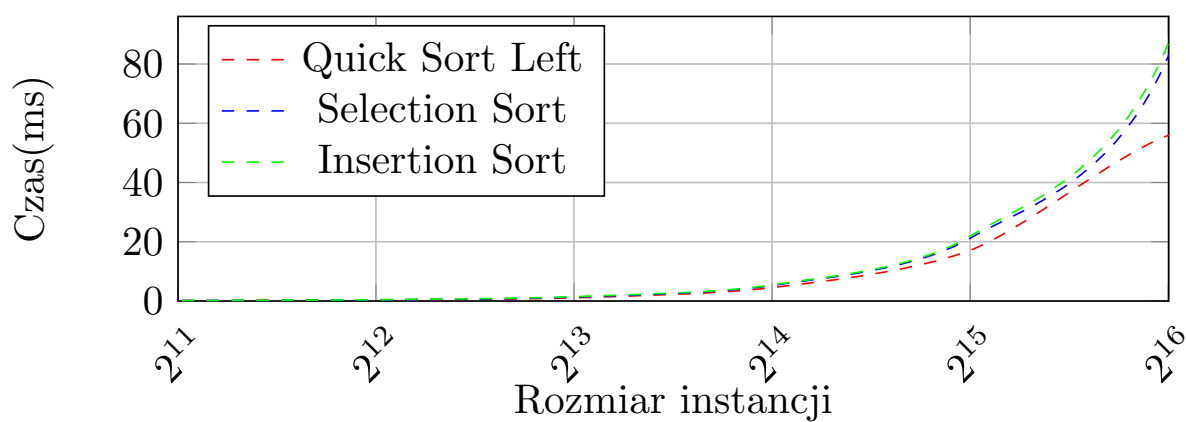


3.5 Dane A-kształtne

Złożoność Obliczeniowa Algorytmy Szybkie



Złożoność Obliczeniowa Algorytmy Wolne



3.6 Komentarz

3.6.1 Insertion Sort

Optymalna złożoność obliczeniowa sortowania przez wstawianie wynosi $O(n)$. W przypadku, gdy tablica jest już posortowana lub prawie posortowana, sortowanie przez wstawianie jest bardzo wydajne, ponieważ wstawianie elementu na właściwe miejsce wymaga jedynie porównania go z jednym elementem z lewej strony. Widać to dobrze przy danych, które są stałe i przy danych rosnących.

Pesymistyczna złożoność obliczeniowa sortowania przez wstawianie wynosi $O(n^2)$. W najgorszym przypadku, gdy tablica jest posortowana w odwrotnej kolejności, każdy nowo wstawiany element będzie musiał przesunąć się na początek tablicy, co wymaga wielu porównań i przesunięć. Z czego wynika wolny czas przy danych malejących.

3.6.2 Selection Sort

Optymalna jak i pesymistyczna złożoność obliczeniowa sortowania przez wybieranie wynosi $O(n^2)$. W obu przypadkach, algorytm selection sort wykonuje to samo łączne porównanie i przestawienie elementów w tablicy. Co czyni go najmniej wydajnym sortowaniem.

3.6.3 Heap Sort

Optymalna złożoność obliczeniowa sortowania przez kopcowanie wynosi $O(n \log n)$. Budowanie kopca i sortowanie w czasie logarytmicznym pozwala na efektywne porządkowanie elementów.

Pesymistyczna złożoność obliczeniowa jest taka sama jak optymistyczna, dzięki temu jest to efektywny algorytm sortowania, który działa stabilnie niezależnie od początkowego ułożenia danych. Na każdym wykresie Heap Sort znajduje się wśród najszybszych algorytmów.

3.6.4 Shell Sort

Pesymistyczna złożoność czasowa dla sortowania Shella z przyrostami Sedgewicka wynosi $O(n^3/2)$, co jest poprawą w stosunku do standardowej wersji Shella. Dzięki zastosowaniu przyrostów zaproponowanych przez Sedgewicka, sortowanie Shella staje się bardziej efektywne dla wielu zestawów danych.

Optymalna złożoność czasowa wynosi $O(n \log n)$, jednak dla niektórych sekwencji przyrostów może być bardziej efektywna od standardowego Shella.

3.6.5 Quick Sort

Dla losowego pivotu zarówno pesymistyczna, jak i optymalna złożoność czasowa wynosi $O(n \log n)$. Dzięki wyborowi losowego pivotu szansa na niekorzystny podział jest zmniejszona, co sprawia, że sortowanie jest bardziej efektywne nawet dla przypadków, gdzie lista jest już posortowana. Zaś przy lewym pivocie pesymistyczna złożoność czasowa wynosi $O(n^2)$, która występuje, gdy lista jest już posortowana lub prawie posortowana.

Optymalna złożoność czasowa to $O(n \log n)$, która występuje w przeciętnym przypadku, gdy pivot jest dobrze wybrany i podział jest równomierny.

4 Podsumowanie

W trakcie realizacji projektu dowiedzieliśmy się wielu cennych rzeczy na temat algorytmów sortowania. Przede wszystkim jak każdy z tych algorytmów działa od podstaw. Każdy z nich ma różną charakterystykę i jest lepiej przystosowany do pewnych zestawów danych. Zrozumieliśmy również znaczenie złożoności obliczeniowej i jak bardzo ma to wpływ na optymalizację czasową naszych programów.